
Ownership and borrowing in Rust

=

clone()

&

&mut



Gábor Takács

<dr.gabortakacs@gmail.com>

Who frees the memory?

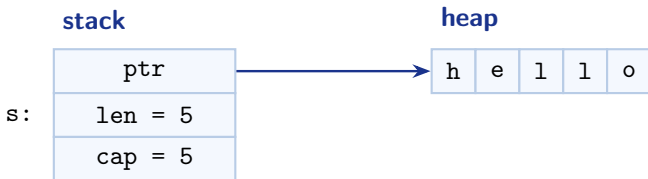
Approach	Who decides	The price
manual free (C, C++)	the programmer	leaks, double free, use-after-free
garbage collector (Java, C#, Go)	the runtime	pauses, memory and CPU overhead
ownership (Rust)	the compiler	you have to say who owns what

Rust decides **at compile time** where every value dies, and writes the `free` into the program for you. There is no garbage collector, no manual `free` and no runtime bookkeeping.

Where does the data live?

```
// A value of *fixed size* lives on the stack.  
let n = 42i32; // 4 bytes on the stack
```

```
// A value that can *grow* keeps its bytes on the heap.  
// The variable holds only metadata on the stack.  
let s = String::from("hello"); // 24 bytes* on the stack  
// 5 bytes on the heap
```



☹️ Somebody must free that heap buffer. Whose job is it?

The three rules of ownership

1. Every value has an **owner** (usually a variable).
2. There can be **only one owner** at a time.
3. When the owner **goes out of scope**, the value is **dropped** and its heap memory is freed.

- ▶ The **compiler** checks all three (and it knows exactly where rule 3 applies).
- ▶ Many of Rust's characteristics follow from these three sentences.

Scope and drop

```
{  
    let s = String::from("inner");  
    println!("{s}");  
} // `s` goes out of scope here:  
  // the heap buffer is freed automatically  
  
println!("{s}");  
// error[E0425]: cannot find value `s` in this scope
```

- ▶ The closing brace is where the free happens. Rust calls this **drop**.
- ▶ You never write it, and you **cannot forget** it.
- ▶ Values are dropped in **reverse order** of their declaration.

Assignment moves ownership

```
let s1 = String::from("hello");  
let s2 = s1; // `ptr`, `len` and `cap` are copied,  
             // the heap buffer is NOT: now `s2` owns it  
  
println!("{s2}"); // works fine  
println!("{s1}"); // error[E0382]: borrow of moved value: `s1`
```

- ▶ Why not simply let both names use the buffer?
Because at the end of the scope **both would free it!** 🐛
- ▶ Rust does not copy the buffer (that would be expensive)
and does not free it twice (that would be a **double free** bug):
it **invalidates the old name** instead.

Copy types and clone()

```
let a = 5;  
let b = a;  
println!("{a} {b}"); // 5 5 (`a` is still usable)
```

- ▶ A **Copy** type owns **nothing on the heap**: the integers, the floats, **bool**, **char** and shared references. For them a “move” is a cheap bitwise copy, and the source variable (a above) stays usable.
- ▶ Your own types need `#[derive(Copy, Clone)]` to be Copy.
- ▶ **String** is not Copy: copying it would mean copying the heap buffer too.

```
let s1 = String::from("hello");  
let s2 = s1.clone(); // now the heap buffer is copied too  
println!("{s1} {s2}");
```

- ▶ `clone()` is **always explicit**, because it can be expensive.

Passing a value moves it (unless it is Copy)

```
fn consume(s: String) {  
    println!("{s}");  
}    // s is dropped here  
  
let s = String::from("hello");  
consume(s);    // s is moved into consume  
println!("{s}");    // error[E0382]: borrow of moved value: s
```

```
fn square(n: i32) -> i32 { n * n }  
  
let n = 7;  
println!("{}", square(n));  
println!("{n}");    // works fine: i32 is Copy
```

- ▶ A function parameter is just another variable, so the same rules apply.
- ▶ Giving ownership back works, but it is clumsy: `let s = f(s);`

&: access without ownership

```
fn length(s: &String) -> usize {  
    s.len()  
}    // only the reference goes out of scope,  
    // the String is not dropped  
  
let s = String::from("hello");  
println!("{}", length(&s));  
println!("{}", s); // s is still the owner
```

- ▶ `&s` creates a **reference**: a pointer to a value someone else owns.
- ▶ Using a reference is called **borrowing**.
The borrower **never frees** anything!

`&mut`: access that can modify

```
fn add_excitement(s: &mut String) {  
    s.push('!');  
}  
  
let mut s = String::from("hello"); // the owner must  
add_excitement(&mut s);             // be mut as well  
add_excitement(&mut s);  
println!("{s}"); // hello!!
```

	Can read	Can write	Can free
the owner	yes	if <code>mut</code>	yes
the reference	yes	if <code>&mut</code>	no

The rules of borrowing

1. At any given time a value can have **either** any number of shared references to it, **or** exactly one mutable reference.
2. A shared reference is **read only**; only `&mut` may modify the value.
3. To take `&mut x`, the owner `x` must itself be declared `mut`.
4. A reference must **always be valid**:
it may never outlive the value it points to.
5. While a value is borrowed, it **cannot be moved or dropped**.

- ▶ Short version: **one writer, or many readers** , but never both.
- ▶ The compiler checks this for you; the part of it that does so is the **borrow checker**.

The rules at work

```
let s = String::from("hello");  
let r1 = &s;           // 2 shared borrows:  
let r2 = &s;           // works fine
```

```
let mut s = String::from("hello");  
let m = &mut s;        // 1 mutable borrow:  
m.push('!');           // works fine (because s is mut)
```

```
let mut s = String::from("hello");  
let r = &s;             // 1 shared and 1 mutable borrow:  
let m = &mut s;  
println!("{r}");       // error[E0502]
```

```
let mut s = String::from("hello");  
let m1 = &mut s;        // 2 mutable borrows:  
let m2 = &mut s;  
m1.push('!');           // error[E0499]
```

Why this exact rule?

A **data race** is two accesses to the same data at the same time, where at least one of them is a **write**.

“One writer or many readers” rules that out, and Rust checks it **at compile time**, for free. This is what makes concurrency in Rust “fearless”.

- ▶ The borrowing rules also prevent single-threaded bugs, such as calling `s.clear()` while a slice of `s` is still alive, or pushing to a collection while you are iterating over it.

A borrow ends at its last use

```
let mut s = String::from("hello");  
let r = &s;  
println!("{r}"); // the last use of r:  
                // the borrow ends HERE  
s.push('!');     // so s can be borrowed mutably again  
println!("{s}");
```

- ▶ A borrow does **not** last until the end of the block, only until its **last use**.
- ▶ So the rules are far less restrictive in practice than they sound at first.
- ▶ If the `println!("{r}")` were moved *after* the push, the same code would no longer compile.

No dangling references

```
fn dangle() -> &String {  
    let s = String::from("hello");  
    &s  
}  
    // s is dropped here, so the reference  
    // would point to freed memory  
// error[E0106]: missing lifetime specifier  
  
fn no_dangle() -> String {    // return the ownership  
    String::from("hello")    // instead  
}
```

- ▶ The analogous code **compiles in C**, and gives a use-after-free bug.
- ▶ In Rust, the program **does not compile**.

&str is a borrowed view of a string

```
let mut s = String::from("hello world");  
let first = &s[0..5]; // borrows a part of s  
println!("{first}");  
  
s.clear();           // error[E0502] IF first is  
println!("{first}"); // still used afterwards
```

- ▶ Clearing `s` would leave `first` pointing at freed memory, so while `first` lives, `s` cannot be borrowed mutably.
- ▶ This is the answer to the question left open in `strings.rs`: the `&` in `&str` borrows the bytes **from their owner**.
- ▶ And `let c = a + &b;` moved `a` because `+` **takes over** its left operand, reusing its buffer instead of allocating a new one.

Therefore: take `&str`, not `&String`

```
fn shout(text: &str) -> String {  
    text.to_uppercase()  
}  
  
let s = String::from("hello");  
shout(&s);           // works with a String  
shout("literal");    // and with a literal  
shout(&s[0..2]);      // and with a slice
```

- ▶ `&String` would accept **only** the first one.
- ▶ **Rule of thumb:** take `&str` as a parameter, return `String` as a result.

What all this buys us

- 😊 no dangling pointers, no use-after-free
- 😊 no double free and no leaks in ordinary code
- 😊 no data races between threads
- 😊 no garbage collector: no pauses, no runtime overhead
- 😊 and all of it is checked **before** the program ever runs
- 😞 the compiler says *no* more often than you would like
- 😞 you have to think about who owns what, even in a ten-line program

Fighting the borrow checker is the normal beginner experience. ;-)

Typical beginner errors

- ☹ using a value after it was moved E0382
- ☹ a shared and a mutable borrow alive at the same time E0502
- ☹ two mutable borrows at the same time E0499
- ☹ writing through a & reference E0596
- ☹ taking `&mut x` where `x` is not `mut` E0596
- ☹ returning a reference to a local value E0106
- ☹ moving a value that is still borrowed E0505
- 😊 the compiler's error messages are very informative
- 😊 `clone()` is always a legitimate way out

Cheat sheet

```
let s2 = s1;           // MOVE: s1 is dead from here
let s2 = s1.clone();   // COPY of the heap buffer too
let b = a;             // a is Copy (i32, f64, bool, char):
                      // a stays alive

fn f(s: String)        { } // takes ownership, drops it
fn f(s: &String)        { } // borrows, read only
fn f(s: &mut String)    { } // borrows, may modify
fn f(s: &str)           { } // borrows any string: prefer this

let r = &s;            // shared borrow: many allowed
let m = &mut s;        // mutable borrow: exactly one,
                      // and s must be `mut`

// EITHER many & OR one &mut, never both
// a borrow ends at its LAST USE
// a reference may never outlive the value it points to
```